

Node.js + TypeScript

RESTful APIs

From JavaScript to typed backend services

Agenda

1. **JavaScript vs TypeScript** – why types matter
2. **Node.js + Express + TypeScript** – setting up a typed server
3. **RESTful APIs** – designing clean HTTP services

Part 1

JavaScript vs TypeScript

What is TypeScript?

TypeScript is a **superset of JavaScript** – all valid JS is valid TS.

	JavaScript	TypeScript
Types	Dynamic (runtime)	Static (compile-time)
Errors caught	At runtime	At compile time
Tooling	Basic	Rich IntelliSense
Runs in browser	Directly	Compiled to JS first
Learning curve	Low	Slightly higher

The Problem with JavaScript

```
// JavaScript – no complaints until it crashes at runtime
function add(a, b) {
  return a + b;
}

add(5, "10");    // Returns "510" – probably not what you wanted
add(5, null);    // Returns 5 – silently wrong
```

TypeScript Catches It Early

```
// TypeScript - error at compile time, before it ever runs
function add(a: number, b: number): number {
  return a + b;
}

add(5, "10");    // ❌ Argument of type 'string' is not assignable to 'number'
add(5, null);    // ❌ Argument of type 'null' is not assignable to 'number'
add(5, 10);      // ✅ Returns 15
```

Bugs are caught **before deployment**, not in production.

Key TypeScript Features

```
// Interfaces – define the shape of objects
interface User {
  id: number;
  name: string;
  email: string;
  role?: "admin" | "user"; // optional, union type
}
```

```
// Type inference – TypeScript figures it out
const greeting = "Hello"; // inferred as string
```

```
// Generics – reusable typed functions
function first<T>(arr: T[]): T {
  return arr[0];
}
```

```
first([1, 2, 3]); // returns number
first(["a", "b", "c"]); // returns string
```

Why Use TypeScript?

- **Fewer runtime bugs** – type errors caught at compile time
- **Better refactoring** – IDE knows all usages of a symbol
- **Self-documenting** – types describe what functions expect and return
- **Team-scale** – essential for large codebases and teams

TypeScript is JavaScript with guard rails.

Part 2

Node.js + Express + TypeScript

Project Setup

```
mkdir my-api && cd my-api
npm init -y
npm install express
npm install -D typescript @types/node @types/express tsx
npx tsc --init
```

Key packages:

- `express` – web framework
- `@types/express` – TypeScript definitions for Express
- `tsx` – run TypeScript directly (no compile step needed in dev)

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",          // compile to modern JS (async/await, etc.)
    "module": "ESNext",         // use ES module syntax (import/export)
    "moduleResolution": "bundler", // resolve modules like a bundler (Vite/tsx)
    "esModuleInterop": true,    // allows default imports from CommonJS packages
    "strict": true,             // enables all strict type checks
    "outDir": "./dist",         // compiled JS output folder
    "rootDir": "./src"          // where your TypeScript source lives
  },
  "include": ["src/**/*"],      // only compile files under src/
  "exclude": ["node_modules", "dist"]
}
```

tsconfig.json

Option	Why it matters
<code>target</code>	Sets the JS version emitted – ES2022 gives you modern features
<code>strict</code>	Catches the most bugs – always enable this
<code>esModuleInterop</code>	Needed to <code>import express from "express"</code> cleanly
<code>moduleResolution: bundler</code>	Works correctly with <code>tsx</code> and modern tooling

package.json Scripts

```
{  
  "type": "module",  
  "scripts": {  
    "dev": "tsx watch src/index.ts",  
    "build": "tsc",  
    "start": "node dist/index.js"  
  }  
}
```

- `dev` – hot-reload development server
- `build` – compile TypeScript to JavaScript
- `start` – run compiled output in production

Basic Express Server - no routes yet

```
// src/index.ts
import express from "express";

const app = express();
app.use(express.json());

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Run with: `npm run dev`

Recommended Project Structure

```
src/  
  index.ts          ← entry point, app setup  
  routes/  
    users.ts        ← route handlers  
  controllers/  
    userController.ts ← business logic  
  models/  
    user.ts          ← types / interfaces  
  middleware/  
    auth.ts          ← authentication middleware  
dist/                ← compiled output (git-ignored)
```

Separating concerns keeps the codebase **maintainable and testable**.

You don't need to set up this folder structure right away but keep this in mind, we'll keep coming back to this as we build our API

Part 3

RESTful APIs - Intro

What is a RESTful API?

REST = Representational State Transfer

An architectural style for building APIs over HTTP.

Two core principles:

- **Platform independence** – any client can call the API via standard HTTP
- **Loose coupling** – client and server evolve independently

REST is not a protocol – it's a set of constraints.

REST Constraints Overview

#	Constraint	One-liner
1	Uniform Interface	Consistent URIs and methods across the API
2	Statelessness	Each request is fully self-contained
3	Layered System	Intermediaries are invisible to client and server
4	Cacheability	Responses declare if they can be cached
5	Client-Server	UI and data storage evolve independently
6	Code on Demand (<i>optional</i>)	Server can send executable code to the client

1 – Uniform Interface

Resources are identified by **URIs**. Clients interact with them using **standard HTTP methods** regardless of the server's internal implementation.

- Every resource has a stable, predictable address (`/api/users/123`)
- The same methods (`GET` , `POST` , `PUT` , `DELETE`) work consistently across all resources
- Enables **loose coupling** – clients don't need to know how the server works

If you know one endpoint, you know how all of them behave.

2 – Statelessness

The server **retains no session state** between requests. Every request must contain all the information needed to process it.

- No server-side sessions – authentication tokens travel with **every request**
- Each request is **atomic** – it either succeeds or fails on its own
- Makes services easier to **scale horizontally** – any server can handle any request

```
GET /api/orders/42 HTTP/1.1
```

```
Authorization: Bearer <token> ← auth sent every time, not stored server-side
```

3 – Layered System

The client **cannot tell** whether it is connected directly to the server or to an intermediary.

Intermediaries can sit transparently between client and server:

- **Load balancers** – distribute traffic across multiple servers
- **Caches** – serve repeated responses without hitting the origin
- **API gateways** – handle auth, rate limiting, routing

Layers can be added, removed, or swapped out without the client knowing.

4 – Cacheability

Responses must declare whether they **can or cannot be cached**.

- Reduces server load and latency for repeated requests
- Servers communicate cacheability via response headers:

```
HTTP/1.1 200 OK  
Cache-Control: max-age=3600      ← client may cache for 1 hour
```

```
HTTP/1.1 200 OK  
Cache-Control: no-store         ← never cache (e.g. sensitive data)
```

Poor caching = unnecessary load. Wrong caching = stale data bugs.

5 – Client-Server Separation

The **UI** (client) and **data storage** (server) are completely decoupled.

- Client only knows about URIs and representations (JSON/XML)
- Server only knows about resources and business logic
- Both sides can **evolve independently** – a new mobile app can use the same API without any server changes

```
graph LR; Browser["Browser (React)"] --- HTTP1["HTTP"] --- Express["Express API"]; Mobile["Mobile (Swift)"] --- HTTP2["HTTP"] --- Express; Express --- SQL["SQL"] --- Postgres["Postgres"]
```

Browser (React) — HTTP — Express API — SQL — Postgres
Mobile (Swift) — HTTP — Express API

6 – Code on Demand (*Optional*)

The server can **temporarily extend client functionality** by sending executable code.

- The client can run code sent by the server (e.g. JavaScript, WebAssembly)
- Enables dynamic updates without client changes
- **Rarely used** in modern REST APIs – security risks and complexity

Examples:

- JavaScript loaded dynamically in a web page (old approach)
- Mobile app downloading and running server-sent scripts (risky)
- WebAssembly modules for compute-intensive operations

Code on Demand is the **only optional** REST constraint – most APIs skip this for security and simplicity.

Exercise 1 - Setup Your API

Useful Resources

- TypeScript Docs
- Express.js Guide
- Node.js + TypeScript Guide